1. Array Methods

What does each method do, and what does it return?

Method	What it does	What it returns
push()	Adds an element to the end of an array.	The new array length.
pop()	Removes the last element from an array.	The removed element, or undefined if the array is empty.
shift()	Removes the first element.	The removed element, or undefined if empty.
unshift()	Adds elements to the beginning.	The new array length.
slice()	Returns a shallow copy of part of an array without changing the original.	A new array.
splice()	Adds, removes, or replaces elements in the original array.	An array containing the removed elements.

Example:

```
const numbers = [10, 20, 30];

console.log(numbers.pop());  // 30
console.log(numbers);        // [10, 20]
```

What does pop() return when the array is empty?

It returns undefined because there is no element to remove.

```
const numbers = [];

console.log(numbers.pop()); // undefined
```

2. const, Arrays, and Objects

Can you change an array's elements or an object's properties when declared with const?

Yes. const prevents reassignment of the variable, but it does not prevent modification of the array or object.

```
const numbers = [1, 2, 3];
numbers[0] = 100;

console.log(numbers); // [100, 2, 3]
const student = { name: "Hazem", age: 20 };
student.age = 21;

console.log(student); // { name: "Hazem", age: 21 }
```

How is modifying a value different from reassigning the variable?

- Modifying: Changing an element or property inside an existing array or object.

- Reassigning: Making the variable refer to a different value.

```
const numbers = [1, 2, 3];

numbers.push(4);       // Valid: modifies the array
// numbers = [5, 6];   // Error: reassigns the variable
```

let allows reassignment, while const does not.

3. Function Declaration vs. Function Expression

What is the difference?

Function declaration:

```
function greet() {
   return "Hello";
}

console.log(greet()); // Hello
```

Function expression using const:

```
const greet = function() {
   return "Hello";
};

console.log(greet()); // Hello
```

A function declaration defines a named function directly. A function expression creates a function and assigns it to a variable.

Can you call each one before the line where it is defined?

A function declaration can be called before its definition because function declarations are hoisted.

```
greet();

function greet() {
   console.log("Hello");
}

// Output: Hello
```

A function expression assigned to const cannot be called before its initialization.

```
// greet(); // ReferenceError

const greet = function() {
   console.log("Hello");
};

greet(); // Output: Hello
```

The variable declared with const is not available for use before its initialization.

4. Implicit Return vs. Explicit Return

What is the difference?

- Explicit return: Uses the return keyword.
- Implicit return: An arrow function returns an expression automatically when written without curly braces.

Explicit return:

```
const add = (a, b) => {
   return a + b;
};

console.log(add(2, 3)); // 5
```

Implicit return:

```
const add = (a, b) => a + b;

console.log(add(2, 3)); // 5
```

How do you return an object using an arrow function?

Wrap the object in parentheses so JavaScript interprets it as an object expression, not a function body.

```
const createStudent = () => ({
   name: "Hazem",
   age: 20
});

console.log(createStudent());
// { name: "Hazem", age: 20 }
```

Without the parentheses, the curly braces would be interpreted as the function body.

5. Parameters and Arguments

What is the difference between a parameter and an argument?

- A parameter is a variable listed in the function definition.
- An argument is the actual value passed to the function when it is called.

```
function greet(name) {  // name is a parameter
   return `Hello ${name}`;
}

greet("Hazem");         // "Hazem" is an argument
```

6. Function Return Values

What does a function return when it has no return statement?

It returns undefined automatically.

```
function sayHello() {
   console.log("Hello");
}

console.log(sayHello());
// Hello
// undefined
```

Does console.log() return the value it displays?

No. console.log() displays a value in the console but returns undefined.

```
const result = console.log("Hello");

console.log(result);
// Hello
// undefined
```